Programming Assignment 4 - Software Pipelining

1. Introduction

In Programming Assignments 1 and 1.5, you implemented GEMM kernels and optimized them using Tensor cores in CUDA. This assignment is the final step in optimizing your kernels. You will use software pipelining in CUDA and (hopefully) make your kernels faster.

2. Software Pipelining

So far, the MMA loop in your kernels looked like this,

1. Load input subtiles from global memory to shared memory
2. Perform MMA using the `mma_m16n8k16_f16_f16_smem_row_col` device function

and while this works, it is not the most efficient way to perform GEMM.

Compute workloads typically have two main pieces: data movement and computation. You load some data from memory, carry out some math on it, and store the results back to memory. To achieve the best performance, you will want to keep your math units running, i.e. your Tensor cores for GEMM. Now, if you profile your kernels, you would see that your Tensor cores are not running at full efficiency because they often have to stall and wait for memory. To address this, you will need to **overlap data movement and computation**. This is where the concept of software pipelining comes into picture.

The CUDA programming model features *pipeline* objects that will let you create and manage a software pipeline where you can **asynchronously** load data from global memory while performing math using the Tensor cores, effectively overlapping memory and compute. This is best explained with an example. See the simplified CUDA code below that we borrowed from the official CUDA documentation.

Here, “producers” refer to the threads that load data from global memory to shared memory, and “consumers” refer to the threads that do math, i.e. MMAs. To keep things simple, all threads in a thread block are both producers and consumers. In each iteration of the for loop, we submit asynchronous memory copies using `cuda::memcpy_async()`. This is a non-blocking function, meaning that threads issue the memory copies, but do not wait for the copies to finish before moving on to the next instruction. Hence, “asynchronous”. It takes 4 parameters: destination address, source address, size in bytes, and the pipeline object. Note that the `cuda::memcpy_async()` function is defined under both `cuda` and `cooperative_groups` namespaces. You should use `cuda::memcpy_async()`. See here for more details.

Next, the threads wait for the memory copies from the **previous stage** to finish before doing MMAs. And thus, we overlap the compute of the current stage with the memory copy of the next stage. The *pipeline* object handles the synchronization implicitly.

The CUDA documentation explains the pipelining in detail and I recommend reading it to learn more. Nevertheless, your final kernel's structure will look very similar to the example below. You just need to fill the gaps.

```
auto pipeline = cuda::make_pipeline();
// Initialize first pipeline stage by submitting a `memcpy_async` to fetch a
whole subtile for the thread block:
pipeline.producer_acquire();
cuda::memcpy_async();
pipeline.producer_commit();

// Pipelined copy/compute:
for (size_t batch = 1; batch < batch_sz; ++batch) {
   // Stage indices for the compute and copy stages:
   size_t compute_stage_idx = (batch - 1) % 2;
   size_t copy_stage_idx = batch % 2;

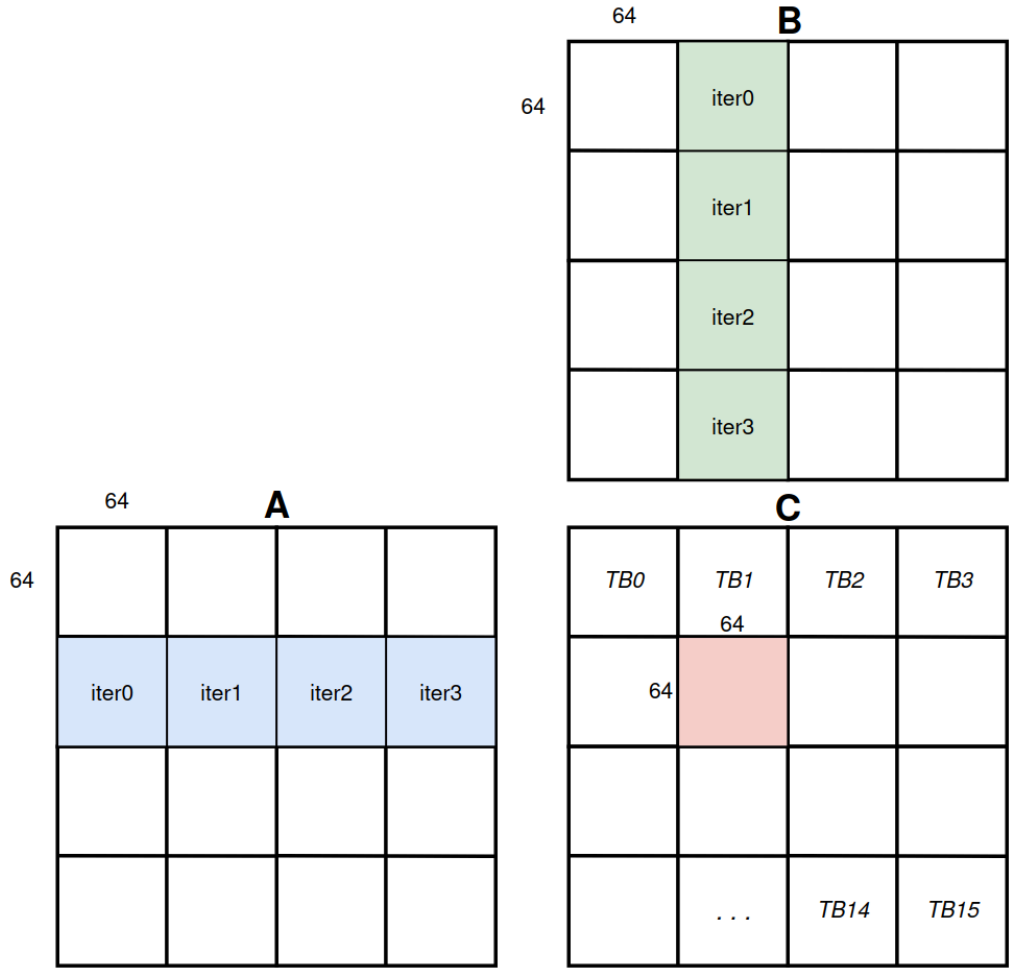
   // Collectively acquire the pipeline head stage from all producer threads:
   pipeline.producer_acquire();
   // Submit async copies to the pipeline's head stage to be
   // computed in the next loop iteration
   cuda::memcpy_async();
   // Collectively commit (advance) the pipeline's head stage
   pipeline.producer_commit();

   // Collectively wait for the operations committed to the
   // previous `compute` stage to complete:
   pipeline.consumer_wait();
   // Computation(MMAs) overlapped with the memcpy_async of the "copy" stage:
   compute();
   // Collectively release the stage resources
   pipeline.consumer_release();
}

// Compute the data fetch by the last iteration
pipeline.consumer_wait();
compute();
pipeline.consumer_release();
```

3. Kernel Design

Similar to PA1.5, you need to think in terms of thread blocks. I recommend launching **4 warps per TB**, and letting each TB calculate a **64x64** tile of the output matrix, as shown in the figure below. In each iteration, fetch a 64x64 tile of A and a 64x64 tile of B. Since you have 128 threads in the TB, that means each thread will load 32 elements of A and 32 elements of B. Once you submit two `cuda::memcpy_async()` calls(one for A and one for B), you can use the `mma_m16n8k16_f16_f16_smem_row_col_64x64()` primitive that we provide. This primitive is similar to the previous ones, but instead, it multiplies two 64x64 matrices stored in shared memory. Keep in mind that this primitive requires exactly 4 warps in the TB that are laid out sequentially, i.e. `dim3(128,1,1)`.



One additional thing you may need to do is to increase the shared memory size for your kernel, and use dynamic shared memory. By default, CUDA allows 49KB of shared memory per TB, but you may need more than that in your code. Add the following code just before your kernel launch,

```
int smemBytes = 65536;
cudaFuncSetAttribute(yourKernelName,
cudaFuncAttributeMaxDynamicSharedMemorySize, smemBytes);
```

This will let the driver know that you need 65536 bytes of shared memory per TB. You also need to declare your shared memory like this in your code,

```
extern __shared__ __align__(128) int8_t smem[];
```

And when you launch the kernel, you pass a third parameter to indicate the size of the shared memory,

```
yourKernelName<2><<<grid, cta, smemBytes>>> ();
```

Simple as that! See this blog post and the documentation for more information.

Here are some tips that might help you,

1. The entire pipelined tensor core kernel can be written in ~70 lines of code. Keep it simple.
2. Do not forget to initialize the shared memory for C to zeroes.
3. You can use `__syncwarp()` and `__synchthreads()` CUDA functions to synchronize threads in a warp or in a block, respectively.
4. CUDA supports `printf()`. You can use it to debug your code, although it is not good practice. Ideally you should use `cuda-gdb`.
5. The CUDA documentation pages have lots of useful information. Make it a habit to browse and read them.

4. Task Description

Your task is to use the MMA wrappers to perform software pipelined GEMM. Use the wrappers to write CUDA kernels for GEMM. Similar to PA1.5, you also need to write the host code to read/initialize the inputs, allocate device memory and launch the kernel. Prepare a 1-page report briefly describing how your host code and the kernel work. Measure the kernel execution time for your new tensor core kernel and compare it to your GEMM kernel from PA1.5. How much faster is your tensor core kernel, if faster at all? You are **not** allowed to use any CUDA libraries(CUTLASS, CuDNN etc.).

5. Constraints

Problem size: **64 <= M,N,K <= 4096**
M, N, K are guaranteed to be integer powers of 2.
Input matrices will be in **fp16** format, i.e. `__half`.
Output matrix will be in **fp32** format, i.e. `float`.

To simplify things, you only need to support **row-major** format for A, and **column-major** format for B. The output matrix **C** will be stored **row-major** in memory. More formally,

{T} x {N} = T

where, **T** means row-major, and **N** means column-major.

6. Materials, Testing and Grading

Similar to the previous assignments, template CUDA(.cu) files and a Makefile are provided in a zip file in Moodle. In the zip file, you will find two CUDA files and a Makefile to build them. The `mma_instrinsics.cuh` file is also included in the zip file. You simply need to fill out the CUDA files.

- `main.cu` is where you read/initialize inputs, allocate device memory, launch kernels and time them. You can compile this file by running "`make main`".
- `launchStudentKernel.cu` has a "`launchStudentKernel()`" function that you will implement, similar to PA1.5. You can simply copy your kernel code from `main.cu`, paste it in the file, and launch it in the `launchStudentKernel()` function. You can compile the code by running "`make pa4_testbench`". This will compile the code, and link it with the testbench binary in the shared directory. It will produce the "pa4_testbench" binary in your directory, which you can run to test your kernels. Example: "`./pa4_testbench 256 256 128`". This will launch the testbench with m256n256k128 problem size where A is row-major and B is column-major. It will invoke your kernel, and a CUTLASS kernel, and compare their outputs. It will print "`[SUCCESS]`" if the outputs match, "`[FAIL]`" otherwise.

We will provide you with multiple sets of small input-output pairs in .txt format to ease testing. Your program can read the inputs, calculate the outputs and compare them with the expected output. You **do not have** to use these input-output references, they are simply provided for convenience. Feel free to generate your own tests and use them.
We will test your program on Hydra servers, so make sure you get the expected outputs there. You will lose points if your code works on your personal computer, but not on Hydra. See here if you are not familiar with ECE Hydra servers.

You can run "`make grade`" to run the grading script which uses the testbench to run several matrix multiplications and checks your results. We will use this exact same script to determine your grades, so you can basically self-grade before you submit. You get 80 points if you pass all the tests, and 20 points for your report, totaling 100.

7. Cheating and LLMs

- You are supposed to do this assignment on your own. You can have discussions with your classmates, but we expect you to write your own code at the end of the day.
- Do not submit AI generated code.

Violation of these rules may result in zero marks and disciplinary action.